

User Guide — Warehouse Safety CV (J&J Capstone)

Table of Contents

1. [Installation & Model Download](#)
 2. [Camera Calibration](#)
 3. [Running the Pipeline](#)
 4. [Output Format](#)
 5. [Violation Events Webhook API](#)
 6. [Life-Saving Rules Reference](#)
 7. [Tuning Thresholds](#)
 8. [Adding a New Rule Module](#)
 9. [Retraining the Detector](#)
 10. [Running the Tests](#)
 11. [Troubleshooting](#)
-

1. Installation & Model Download

Requirements: Python 3.10+, NVIDIA GPU with CUDA 12.4 (CPU execution supported with `--no-pose`).

```
python -m venv .venv
.venv\Scripts\activate # Windows – use source .venv/bin/activate on Linux/macOS
pip install -r requirements.txt
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu124 # skip on Colab
```

1.1 Download Pre-trained Weights

Download the fine-tuned model checkpoint from Google Drive:

- **Download Link:** [Google Drive — rfdetr real model weights](#)
- Place the downloaded `checkpoint_best_ema.pth` file into the repository at:
`models/rfdetr_real/checkpoint_best_ema.pth`

1.2 Verify Installation

Run the automated test suite to confirm the environment is configured correctly:

```
python -m pytest -q # 91 tests, ~6 s
```

2. Camera Calibration

Calibration maps camera pixel coordinates to real-world floor metres. It is performed once per camera view during setup (~10 minutes).

2.1 Pick floor reference points

```
python -m scripts.pick_calibration_points data/calibration/cam1_ref.jpg --camera-id cam1
```

An interactive window opens. Click **at least 4 points on the floor** (not on elevated objects or in mid-air) and enter their real-world (x, y) coordinates in metres when prompted. The tool writes `data/calibration/cam1.json`.

Point Ordering: `image_points[i]` must correspond directly to `floor_points[i]` in the JSON.

2.2 Validate calibration

Measure two reference points on the floor with a laser/tape measure and verify homography accuracy:

```
python -m src.geometry data/calibration/cam1.json --pair X1 Y1 X2 Y2 TRUE_METRES
# Example:
python -m src.geometry data/calibration/cam1.json --pair 412 880 1180 875 6.0
```

Calibration is verified when distance error is < 10%.

2.3 Set the vehicle length

Edit `vehicle_length_m` in the calibration JSON to reflect the site's forklift specifications (e.g. 2.7 m for reach trucks, 4.5 m for counterbalance trucks). Rule 3 proximity zones scale directly with this value.

2.4 Add walkway polygons (for Rule 4)

Add a `walkways` array of floor-coordinate polygons to the calibration JSON to designate compliant pedestrian zones:

```
{
  "camera_id": "cam1",
  "image_points": [...],
  "floor_points": [...],
  "vehicle_length_m": 2.7,
  "walkways": [
    [[0.0, 0.0], [10.0, 0.0], [10.0, 3.0], [0.0, 3.0]]
  ]
}
```

Rule 4 evaluates pedestrian adherence against these configured zones.

3. Running the Pipeline

3.1 Inspect class IDs

Inspect class IDs from the dataset configuration:

```
python -m scripts.discover_class_ids data/dataset
```

3.2 Execute

```
python -m src.run_pipeline \
  --video data/raw_videos/clip.mp4 \
  --calib data/calibration/cam1.json \
  --weights models/rfdetr_real/checkpoint_best_ema.pth \
  --worker-weights models/phone/checkpoint_best_ema.pth \
  --person-id 2 --forklift-id 1
```

3.3 CLI Options

Flag	Default	Purpose
--video	(required)	Path to input video file
--calib	(required)	Path to camera calibration JSON
--weights	(required)	Path to checkpoint_best_ema.pth , or "coco" for pretrained baseline
--worker-weights	None	Path to worker phone/PPE RF-DETR checkpoint (src/worker_detector.py)
--outdir	outputs	Destination root directory for events/ and videos/
--device	cuda	Compute device (cuda or cpu)
--person-id	2	Category ID for "person" in detection model
--forklift-id	1	Category ID for "forklift" in detection model
--threshold	0.5	Detection confidence threshold
--no-pose	off	Disable pose estimation for accelerated throughput
--max-frames	None	Process up to N frames (useful for quick verification)
--webhook-url	None	HTTP endpoint to publish violation events to
--webhook-token	None	Optional Bearer authentication token for webhook requests

The pipeline processes at effective `PROC_FPS = 10` (every 3rd frame of 30 fps video), configurable in `CFG['PROC_FPS']` in `src/rules.py`.

4. Output Format

4.1 outputs/events/events.jsonl

Records violation **episodes** as structured JSON lines:

```
{
  "rule": 3,
  "person_track": 0,
  "vehicle_track": 1,
  "distance_m": 2.21,
  "threshold_m": 8.1,
  "vehicle_speed_ms": 0.73,
  "event_id": "evt_00000",
  "camera_id": "cam1",
  "video": "clip.mp4",
  "start_s": 0.47,
  "end_s": 3.67,
  "duration_s": 3.20,
  "peak_s": 1.14,
  "frames": 32,
  "timestamp_s": 1.14,
  "evidence_frame": "outputs/events/evt_00000_rule3.jpg"
}
```

Field	Meaning
rule	Rule identifier (1, 3, 4, or 5)
person_track / vehicle_track / driver_track	ByteTrack track IDs of involved entities
distance_m	(Rule 3) Minimum separation distance recorded in floor metres
threshold_m	(Rule 3) Proximity threshold applied ($3 \times \text{vehicle_length_m}$)
vehicle_speed_ms	(Rule 3) Vehicle speed at peak event severity
seconds_outside	(Rule 5) Maximum continuous keypoint excursion duration
start_s / end_s / duration_s	Episode time boundaries in seconds
peak_s	Timestamp corresponding to the highest violation severity
evidence_frame	Relative path to the saved evidence image with faces blurred

4.2 outputs/videos/annotated.mp4

Rendered video overlay with entity bounding boxes, track IDs, driver identification tags (`drv`), and active rule alerts.

4.3 Evidence Frames

High-resolution JPEG images (`outputs/events/evt_NNNNN_ruleR.jpg`) captured at peak violation severity with privacy-preserving face blurring applied.

5. Violation Events Webhook API

When a violation episode concludes, the pipeline publishes the complete event record via HTTP POST to the configured webhook endpoint. Episodes conclude once the condition remains clear for the 2.0-second cooldown

window, ensuring `duration_s` and `peak_s` are fully aggregated.

5.1 Configuration

Supply the webhook URL and optional auth token via CLI flags:

```
python -m src.run_pipeline \  
  --video data/raw_videos/clip.mp4 \  
  --calib data/calibration/cam1.json \  
  --webhook-url https://api.yourdomain.com/v1/warehouse-events \  
  --webhook-token YOUR_BEARER_TOKEN
```

Or configure via environment variables: - `VIOLATION_WEBHOOK_URL=https://api.yourdomain.com/v1/warehouse-events`
- `VIOLATION_WEBHOOK_TOKEN=YOUR_BEARER_TOKEN`

5.2 Payload Schema

Deliveries send JSON payloads with `Content-Type: application/json`:

```
{  
  "event_id": "evt_00000",  
  "rule": 3,  
  "camera_id": "cam1",  
  "video": "clip.mp4",  
  "start_s": 0.47,  
  "end_s": 3.67,  
  "duration_s": 3.20,  
  "peak_s": 1.14,  
  "frames": 32,  
  "timestamp_s": 1.14,  
  "published_at": "2026-08-18T21:15:00Z",  
  "evidence_frame": "outputs/events/evt_00000_rule3.jpg",  
  "evidence_frame_url": "https://cv-host.local/evidence/evt_00000_rule3.jpg",  
  "details": {  
    "person_track": 0,  
    "vehicle_track": 1,  
    "distance_m": 2.21,  
    "threshold_m": 8.1,  
    "vehicle_speed_ms": 0.73  
  }  
}
```

5.3 Endpoint Requirements

- **Method & Headers:** POST request with `Content-Type: application/json` and `Authorization: Bearer <token>` when configured.
- **Response Handling:** Return `200 OK` or `204 No Content`. The pipeline retries on 5xx errors or network timeouts with exponential backoff.
- **Idempotency:** `event_id` provides a unique identifier per episode for deduplication.
- **Rule-Based Routing:** Route incoming events using the `rule` field:

rule	Event Type	Recommended Routing
3	Working vehicle proximity breach	Real-time safety supervisor pager / floor alert
4	Pedestrian off-walkway	Floor monitor dashboard / supervisor log
5	Driver body outside cab	Real-time safety supervisor pager
1	Mobile device / distraction	Async review queue

6. Life-Saving Rules Reference

Rule	Description	Trigger Condition
1 — No mobile phone / distracting device	Detects sustained device usage near the ear	Wrist-to-head distance ratio < 0.6 for > 2.0 s
2 — Pre-use inspection checklist	Operational procedure verification	Handled via digital/paper pre-shift logs (non-vision)
3 — Pedestrian proximity to working vehicle	Enforces safe exclusion zone around moving forklifts	Pedestrian floor position < 3 × vehicle_length_m from a working forklift
4 — Pedestrians keep to designated walkways	Enforces pedestrian walkway lane compliance	Pedestrian outside configured walkway polygons for > 1.0 s
5 — Driver body inside vehicle cab	Enforces driver containment within vehicle cabin	Driver keypoints (nose, shoulders, wrists) outside cab inset for > 1.5 s

Key Rule Details

- Rule 3 (Proximity):** The safety boundary is calculated as 3 × vehicle_length_m . The forklift operator is automatically identified via velocity co-movement and excluded from triggering pedestrian proximity events.
- Rule 4 (Walkways):** Evaluates non-driver pedestrian positions against the walkways polygons defined in the calibration JSON.
- Rule 5 (Cab Containment):** Evaluates upper-body keypoints against the cab boundaries defined in CFG['R5_CAB_FRACTIONS'] . Head excursions during reversing maneuvers are tracked according to standard workplace safety requirements.

7. Tuning Thresholds

All operational thresholds are defined in the CFG dictionary in [src/rules.py](#) and can be adjusted without retraining:

```
CFG = {
    'PROC_FPS':          10,      # Must match ByteTrackTracker(frame_rate=...)
    'MOVING_MS':         0.3,     # m/s – vehicle is considered "working" above this speed
    'RECENT_MOVE_S':     5.0,     # seconds – vehicle that moved recently remains in "working" state
    'R3_VEHICLE_LENGTHS': 3.0,    # danger radius multiplier (× vehicle_length_m)
    'DRIVER_OVERLAP':    0.6,     # min fraction of person box inside vehicle box for driver candidate
    'DRIVER_VEL_MATCH_MS': 0.5,   # m/s – max velocity delta to confirm driver co-movement
    'R5_CAB_FRACTIONS':  (0.15, 0.35, 0.15, 0.0), # cab inset fractions: (left, top, right, bottom)
    'R5_MIN_S':          1.5,     # seconds outside cab before Rule 5 event triggers
    'R4_MIN_S':          1.0,     # seconds off-walkway before Rule 4 event triggers
    'R1_WRIST_HEAD_RATIO': 0.6,   # wrist-to-head / shoulder-width ratio threshold
    'R1_MIN_S':          2.0,     # sustained duration before Rule 1 event triggers
    'KPT_CONF':          0.5,     # minimum RTMPose keypoint confidence threshold
}
```

Duration parameters (`_MIN_S`) provide temporal filtering to prevent transient false alerts while ensuring persistent events are promptly logged.

8. Adding a New Rule Module

Refer to [INTEGRATION.md](#) for detailed architectural patterns and worked examples.

Option A — Consume `events.jsonl` (External)

```
import json
events = [json.loads(line) for line in open('outputs/events/events.jsonl')]
```

Suitable for dashboarding, reporting, and asynchronous notifications.

Option B — Inline Rule Module (Pipeline)

1. **Implement Rule Logic:** Add rule evaluation in `src/rules.py` returning event dictionaries with `{'rule': N, ...}`.
2. **Register Event Schema:** Register rule fields in `KEY_FIELDS` (and `SEVERITY`) in `src/events.py`.
3. **Invoke in Pipeline Loop:** Integrate rule call inside `src/run_pipeline.py`.
4. **Unit Test:** Add test coverage in `tests/`.

New custom rules should start numbering from **6** (rules 1–5 are reserved).

TrackedObject Interface

```
@dataclass
class TrackedObject:
    track_id: int          # Persistent track identifier across frames
    class_id: int          # Category ID (person or forklift)
    box: tuple             # (x1, y1, x2, y2) bounding box in pixel coordinates
    floor_xy: tuple        # (x, y) ground contact point projected in floor metres
    keypoints: np.ndarray  # (17, 3) keypoint array [x, y, conf] for persons (None for vehicles)
    worker_detections: dict # {'phone': [...], 'helmet': [...], 'vest': [...]} from worker detector
```

9. Retraining the Detector

The pre-trained RF-DETR model (`models/rfdetr_real/checkpoint_best_ema.pth` , mAP50:95: 0.820) provides out-of-the-box detection for warehouse personnel and forklifts.

Dataset Composition

Dataset	Split (Train / Valid)	License
forklift2-z6zww v5	3,192 / 275	CC BY 4.0
forklift-and-human v2	1,361 / 392	CC BY 4.0
NVIDIA SDG-Warehouse	1,333 / 720	OpenMDW 1.1

Training Workflow

```
export ROBOFLOW_API_KEY=<key>
python -m scripts.fetch_roboflow --url https://universe.roboflow.com/pdf-ih16p/forklift2-z6zww/dataset/5
python -m scripts.fetch_roboflow --url https://universe.roboflow.com/hitsz/forklift-and-human/dataset/2
python -m scripts.fetch_sdg --scenario nearmiss --shards 1 2 3 4
python -m scripts.sdg_to_coco --root data/sdg --out data/dataset_v2
python -m scripts.real_to_coco --root data/real/forklift2-z6zww_v5 data/real/forklift-and-human_v2 --merge-v
python -m scripts.train_detector --dataset-dir data/dataset_v3 --output-dir models/rfdetr_new --epochs 15
```

10. Running the Tests

Execute the automated test suite:

```
python -m pytest -q # 91 tests, ~6 s
```

Includes unit tests for all rule state machines, driver association, `CameraGeometry` , `EventAggregator` , and end-to-end synthetic pipeline validation.

Precision & Recall Scoring

```
python -m scripts.score_events --events outputs/events/events.jsonl --truth data/validation/ground_truth.js
```

11. Troubleshooting

Symptom	Likely Cause	Resolution
Video failed to open	Incorrect file path or missing codec	Verify the video path and ensure standard MP4/H.264 format
Detections not appearing	Dataset class ID mismatch	Run <code>python -m scripts.discover_class_ids <dataset_path></code> and pass <code>--person-id</code> and <code>--forklift-id</code>
Low tracking stability	Frame rate configuration mismatch	Ensure <code>ByteTrackTracker(frame_rate=...)</code> matches <code>CFG['PROC_FPS']</code>
Inaccurate floor distance calculations	Calibration point ordering	Verify that <code>image_points[i]</code> corresponds to <code>floor_points[i]</code> and re-run <code>--pair</code> validation
Calibration homography fails to solve	Collinear calibration points	Select floor points spanning a 2D quadrilateral rather than a single line
New rule fires continuously	Rule key unregistered	Add the new rule key to <code>KEY_FIELDS</code> in <code>src/events.py</code>
Noisy pose keypoints	Low keypoint confidence	Use <code>valid(kp[i])</code> from <code>src/pose_utils.py</code> to filter detections below <code>CFG['KPT_CONF']</code>
Import error for RF-DETR	Package version mismatch	Ensure <code>rfdetr>=1.9.0,<2.0.0</code> is installed per <code>requirements.txt</code>
<code>sv.ByteTrack</code> not found	<code>supervision</code> version update	<code>trackers.ByteTrackTracker</code> is used by default in <code>src/run_pipeline.py</code>

For questions, contact the capstone team.